TamboUI provides a comprehensive set of widgets for building terminal UIs.

Widget Overview

Widget	Type	Description
Block	Stateless	Container with borders, titles, and padding
Paragraph	Stateless	Multi-line text with wrapping and alignment
ListWidget	Stateful	Scrollable list with selection
Table	Stateful	Grid with rows, columns, and selection
Tabs	Stateful	Tab bar with selection
Gauge	Stateless	Progress bar with percentage
LineGauge	Stateless	Horizontal line progress indicator
Sparkline	Stateless	Mini line chart for data series
BarChart	Stateless	Vertical bar chart
Chart	Stateless	Line and scatter plots with axes

Widget	Type	Description
Canvas	Stateless	Drawing surface with shapes
Calendar	Stateless	Monthly calendar with date styling
Spinner	Stateful	Animated loading indicator
Scrollbar	Stateful	Visual scroll position indicator
TextInput	Stateful	Single-line text input
TextArea	Stateful	Multi-line text input
Checkbox	Stateful	Toggleable checkbox
Toggle	Stateful	On/off toggle switch
Select	Stateful	Inline select with navigation indicators
Tree	Stateful	Hierarchical tree view with expand/collapse
WaveText	Stateful	Animated wave brightness text effect
Logo	Stateless	Renders the Tamboui logo
ErrorDisplay	Stateless	Displays error information with stack trace
Clear	Stateless	Clears an area (for widget layering)

Container Widgets

Block

A `Block` is the most fundamental container widget. It can have borders, titles, and padding.

```
Block block = Block.builder()
    .title("My Title")
    .borders(Borders.ALL)
    .borderType(BorderType.ROUNDED)
    .borderStyle(Style.EMPTY.fg(Color.CYAN))
    .padding(new Padding(1, 2, 1, 2))  // top, right, bottom, left
    .build();

// Render the block
block.render(area, buffer);

// Get the inner area (after borders and padding)
Rect inner = block.inner(area);
```

Border types:

- `BorderType.PLAIN` - Simple lines (`─` , `│`)
- `BorderType.ROUNDED` - Rounded corners (`╭` , `╮` , `╰` , `╯`)
- `BorderType.DOUBLE` - Double lines (`═` , `╫`)
- `BorderType.THICK` - Thick lines (`━` , `┃`)

Clear

Clears an area by filling it with spaces. Useful before rendering overlays:

```
Clear.INSTANCE.render(dialogArea, buffer);
// Then render your dialog on top
```

For layout widgets (Columns, Grid, Dock, Stack, Flow), see Layouts Reference.

Text Widgets

Paragraph

Displays multi-line text with wrapping and alignment:

```
Paragraph paragraph = Paragraph.builder()
    .text(Text.from("This is a paragraph of text that can wrap across multiple lin
    .overflow(Overflow.WRAP_WORD)
    .alignment(Alignment.LEFT)
    .block(Block.builder()
        .title("Description")
        .borders(Borders.ALL)
        .build())
    .build();

paragraph.render(area, buffer);
```

Wrap modes:

- `Wrap.NONE` - No wrapping, text is clipped
- `Wrap.CHARACTER` - Wrap at any character
- `Wrap.WORD` - Wrap at word boundaries

Selection Widgets

ListWidget

A low-level scrollable list widget with selection. This is part of the widgets layer and requires manual state management with `ListState`.

> **For Toolkit DSL users**: Use `ListElement` instead (via `list()` factory method). It manages state internally and accepts any `StyledElement` as items. See ListWidget vs ListElement.

```
// Create state
ListState state = new ListState();
state.select(0);  // Select first item

// Build and render
ListWidget list = ListWidget.builder()
    .items(
        ListItem.from("First item"),
        ListItem.from("Second item"),
        ListItem.from("Styled item").style(Style.EMPTY.bold())
    )
    .highlightStyle(Style.EMPTY.reversed())
    .highlightSymbol(">> ")
    .block(Block.builder()
        .title("Select an Item")
        .borders(Borders.ALL)
        .build())
    .build();

list.render(area, buffer, state);
```

State methods:

- `state.select(index)` - Select specific index
- `state.selectNext(totalItems)` - Move selection down
- `state.selectPrevious()` - Move selection up
- `state.selectFirst()` - Jump to first item
- `state.selectLast(totalItems)` - Jump to last item

ListWidget vs ListElement

TamboUI provides two list implementations at different API levels:

Feature	ListWidget (widgets layer)	ListElement (toolkit layer)
State management	External `ListState` required	Internal state, no external state needed

Feature	ListWidget (widgets layer)	ListElement (toolkit layer)
Item types	`ListItem` only (text + style)	Any `StyledElement` (text, rows, panels, etc.)
Usage	Direct widget rendering	Toolkit DSL fluent API
Best for	Custom TuiRunner apps, fine-grained control	ToolkitApp, declarative UIs

ListWidget is for low-level control when you need to manage state externally or are building custom rendering with `TuiRunner`.

ListElement is for Toolkit DSL users who want:

- Internal state management (no `ListState` needed)
- Rich content (any `StyledElement` as items, not just text)
- Automatic event handling (UP/DOWN navigation built-in)
- Seamless integration with other toolkit elements

```
// Toolkit DSL - ListElement with rich content
list()
    .add(row(text("*").yellow(), text(" Featured Item")))
    .add(text("Plain Item"))
    .add(panel("Nested Panel", text("Content")).rounded())
    .highlightColor(Color.CYAN)
    .autoScroll();
```

Table

A data table with rows and columns:

```
// Create table
Table table = Table.builder()
    .header(Row.from("Name", "Age", "City"))
```

```
    .rows(
        Row.from("Alice", "30", "NYC"),
        Row.from("Bob", "25", "LA"),
        Row.from("Charlie", "35", "Chicago")
    )
    .widths(
        Constraint.percentage(40),
        Constraint.length(10),
        Constraint.fill()
    )
    .highlightStyle(Style.EMPTY.bg(Color.DARK_GRAY))
    .block(Block.builder()
        .title("Users")
        .borders(Borders.ALL)
        .build())
    .build();

// Create and use state
TableState state = new TableState();
state.select(0);
table.render(area, buffer, state);
```

Tabs

A tab bar for navigation:

```
Tabs tabs = Tabs.builder()
    .titles("Home", "Settings", "About")
    .highlightStyle(Style.EMPTY.bold().fg(Color.CYAN))
    .divider(" | ")
    .block(Block.builder()
        .borders(Borders.BOTTOM_ONLY)
        .build())
    .build();

TabsState state = new TabsState();
state.select(0);
tabs.render(area, buffer, state);
```

Tree

A hierarchical tree view with expand/collapse, keyboard navigation, and support for custom node rendering. Use `TreeElement` (toolkit layer) with `TreeNode` for the data model.

```
// Create tree nodes with data model
TreeNode<FileInfo> src = TreeNode.of("src", new FileInfo("src", FileType.DIRECTORY
    .add(TreeNode.of("main", new FileInfo("main", FileType.DIRECTORY))
        .add(TreeNode.of("App.java", new FileInfo("App.java", FileType.JAVA, 2048)
        .expanded())
    .expanded();

// Create tree with custom node renderer
tree(src, docs, build)
    .title("Project Files")
    .rounded()
    .highlightColor(Color.CYAN)
    .scrollbar()
    .nodeRenderer(node -> row(
        text(node.data().icon() + " "),
        text(node.label()).bold(),
        spacer(),
        text(node.data().formattedSize()).dim()
    ));
```

Model/View Separation

TreeElement encourages clean separation between data and presentation:

Model (TreeNode + Data): Contains only data, no rendering logic.

```
// Pure data model - no view concerns
record FileData(String name, FileType type, long size, Status status) {
    String icon() { /* data-derived */ return ""; }
    String formattedSize() { /* data formatting */ return ""; }
}

FileData data = new FileData("App.java", FileType.JAVA, 2048, Status.OK);
// TreeNode holds the data
TreeNode<FileData> node = TreeNode.of("App.java", data).leaf();
```

View (nodeRenderer): Transforms data into styled elements.

```
// Rendering logic separated from model
tree(node).nodeRenderer(n -> {
    var info = n.data();
    return row(
        text(info.icon() + " "),
        text(info.name()).fg(info.statusColor()),
        spacer(),
        text(info.formattedSize()).dim()
    );
});
```

This separation allows:

- **Reusable models**: Same `TreeNode<T>` data can be rendered differently in different contexts
- **Testable logic**: Data transformations can be unit tested without UI
- **Flexible presentation**: Change rendering without modifying data structures

Keyboard Navigation

Key	Action
↑/↓	Move selection up/down
→	Expand node or move to first child
←	Collapse node or move to parent
Enter/Space	Toggle expand/collapse
Home/End	Jump to first/last visible node
Page Up/Down	Scroll by viewport height

Lazy Loading

Load children on-demand for large trees:

```
TreeNode.of("Large Directory", dirInfo)
    .childrenLoader(() -> loadChildrenFromDisk(dirInfo.name()));
```

The loader is called once when the node is first expanded.

CSS Styling

Style trees with CSS, including the `:selected` pseudo-class:

```
/* Guide characters */
TreeElement-guide {
    color: #45475a;
}

/* Selected node styling */
TreeElement-node:selected {
    color: #89dceb;
    background: #313244;
    bold: true;
}

/* Scrollbar */
TreeElement-scrollbar-thumb {
    color: #89b4fa;
}
```

CSS selectors:

- `TreeElement` - the container
- `TreeElement-node` - each tree node
- `TreeElement-node:selected` - the selected node
- `TreeElement-guide` - guide/branch characters
- `TreeElement-scrollbar-thumb` - scrollbar thumb
- `TreeElement-scrollbar-track` - scrollbar track

Data Visualization

Gauge

A progress bar showing completion percentage:

```
Gauge gauge = Gauge.builder()
    .ratio(0.75)  // 75% complete
    .label("Loading...")
    .gaugeStyle(Style.EMPTY.fg(Color.GREEN))
    .block(Block.builder()
        .title("Progress")
        .borders(Borders.ALL)
        .build())
    .build();

gauge.render(area, buffer);
```

LineGauge

A compact single-line progress indicator:

```
LineGauge lineGauge = LineGauge.builder()
    .ratio(0.5)
    .label("50%")
    .lineSet(LineGauge.THICK)
    .filledStyle(Style.EMPTY.fg(Color.CYAN))
    .unfilledStyle(Style.EMPTY.fg(Color.DARK_GRAY))
    .build();

lineGauge.render(area, buffer);
```

Sparkline

A mini chart showing data trends:

```
int[] data = {1, 4, 2, 8, 5, 7, 3, 6};

Sparkline sparkline = Sparkline.builder()
    .data(data)
    .foreground(Color.CYAN)
    .block(Block.builder()
        .title("CPU Usage")
        .borders(Borders.ALL)
        .build())
    .build();

sparkline.render(area, buffer);
```

BarChart

Vertical bar chart for comparing values:

```
BarChart barChart = BarChart.builder()
    .data(BarGroup.of(
        Bar.of(80, "Mon"),
        Bar.of(95, "Tue"),
        Bar.of(60, "Wed"),
        Bar.of(75, "Thu"),
        Bar.of(90, "Fri")
    ))
    .barWidth(5)
    .barGap(1)
    .barColor(Color.GREEN)
    .labelColor(Color.DARK_GRAY)
    .block(Block.builder()
        .title("Weekly Sales")
        .borders(Borders.ALL)
        .build())
    .build();

barChart.render(area, buffer);
```

Chart

Line and scatter plots with axes:

```java
// Create datasets
Dataset dataset1 = Dataset.builder()
    .name("Series A")
    .data(new double[][] {
        {0, 0},
        {1, 2},
        {2, 1},
        {3, 4},
        {4, 3}
    })
    .marker(Dataset.Marker.BRAILLE)
    .style(Style.EMPTY.fg(Color.CYAN))
    .build();

// Build chart
Chart chart = Chart.builder()
    .datasets(dataset1)
    .xAxis(Axis.builder()
        .title("X Axis")
        .bounds(0, 5)
        .build())
    .yAxis(Axis.builder()
        .title("Y Axis")
        .bounds(0, 5)
        .build())
    .block(Block.builder()
        .title("Line Chart")
        .borders(Borders.ALL)
        .build())
    .build();

chart.render(area, buffer);
```

Marker types:

- `Marker.DOT` - Single dots
- `Marker.BRAILLE` - Braille patterns for higher resolution
- `Marker.BLOCK` - Block characters

Canvas

A drawing surface for custom graphics:

```
Canvas canvas = Canvas.builder()
    .xBounds(-10, 10)
    .yBounds(-10, 10)
    .marker(Marker.BRAILLE)
    .paint(ctx -> {
        // Draw shapes
        ctx.draw(new Circle(0, 0, 5, Color.RED));
        ctx.draw(new Line(-5, -5, 5, 5, Color.GREEN));
        ctx.draw(new Rectangle(-3, -3, 6, 6, Color.BLUE));
    })
    .block(Block.builder()
        .title("Drawing")
        .borders(Borders.ALL)
        .build())
    .build();

canvas.render(area, buffer);
```

Available shapes:

- `Circle` - Circle with center and radius
- `Line` - Line between two points
- `Rectangle` - Rectangle with position and size
- `Points` - Collection of individual points

Calendar

A monthly calendar widget:

```
CalendarEventStore events = CalendarEventStore.today(Style.EMPTY.bold().fg(Color.C

Monthly calendar = Monthly.of(LocalDate.now(), events)
    .showMonthHeader(Style.EMPTY.bold())
    .showWeekdaysHeader(Style.EMPTY.fg(Color.CYAN))
    .showSurrounding(Style.EMPTY.dim())
    .block(Block.bordered());

calendar.render(area, buffer);
```

Animated Widgets

Spinner

An animated loading indicator that cycles through frame characters:

```
// Create spinner with built-in style
Spinner spinner = Spinner.builder()
    .spinnerStyle(SpinnerStyle.DOTS)
    .style(Style.EMPTY.fg(Color.CYAN))
    .build();

// Or with custom frames
Spinner spinnerCustom = Spinner.builder()
    .frames("*", "+", "x", "+")
    .build();

// Or with a custom frame set
Spinner spinnerFrameSet = Spinner.builder()
    .frameSet(SpinnerFrameSet.of("a", "b", "c", "d"))
    .build();

// Create state and render
SpinnerState state = new SpinnerState();
spinner.render(area, buffer, state);

// Advance state on each tick
state.advance();
```

Built-in styles:

- `SpinnerStyle.DOTS` - Braille dot pattern
- `SpinnerStyle.LINE` - Classic `-\|/`
- `SpinnerStyle.ARC` - Quarter-circle characters
- `SpinnerStyle.CIRCLE` - Clock-position circle
- `SpinnerStyle.BOUNCING_BAR` - Bouncing bar `[=== ]`
- `SpinnerStyle.TOGGLE` - Two-state toggle
- `SpinnerStyle.GAUGE` - Horizontal block fill (| ▌▌▌▌▌▌▌)

- `SpinnerStyle.VERTICAL_GAUGE` - Vertical bar growth (▁▂▃▅▆▇)
- `SpinnerStyle.ARROWS` - Rotating arrow directions
- `SpinnerStyle.CLOCK` - Clock face emoji (🕐🕑…)
- `SpinnerStyle.MOON` - Moon phase emoji (🌑🌒…)
- `SpinnerStyle.SQUARE_CORNERS` - Rotating square corners
- `SpinnerStyle.GROWING_DOTS` - Braille dots filling
- `SpinnerStyle.BOUNCING_BALL` - Bouncing ball in braille

CSS Properties

Spinner style and frames can be configured via CSS:

```
/* Use a predefined style */
.loading-spinner {
    spinner-style: line;
    color: cyan;
}

/* Use custom animation frames */
.custom-spinner {
    spinner-frames: "⠈" "⠐" "⠠" "⠄" "⠤" "⠥" "⠭" "⠯";
}

/* spinner-frames overrides spinner-style when both are set */
.fancy-spinner {
    spinner-style: dots;
    spinner-frames: "*" "+" "x" "+";  /* This takes precedence */
}
```

> Programmatic styles set via `spinnerStyle()`, `frameSet()`, or `frames()` take precedence over CSS styles.

> **For Toolkit DSL users**: Use `SpinnerElement` via `spinner()` factory methods. It manages state internally and advances automatically on each render. See Using Widgets with Toolkit DSL.

WaveText

An animated text widget with a wave brightness effect:

```
// Dark shadow moving through bright text (default)
WaveText wave = WaveText.builder()
    .text("Processing...")
    .color(Color.CYAN)
    .build();

// Bright peak moving through dim text (inverted)
WaveText waveInverted = WaveText.builder()
    .text("Loading...")
    .color(Color.YELLOW)
    .inverted(true)
    .build();

// Back-and-forth oscillation instead of looping
WaveText waveOscillate = WaveText.builder()
    .text("Thinking...")
    .mode(WaveText.Mode.OSCILLATE)
    .build();

// Multiple peaks for dynamic effect
WaveText waveMulti = WaveText.builder()
    .text("Working hard...")
    .color(Color.GREEN)
    .peakCount(2)
    .peakWidth(5)
    .speed(1.5)
    .build();

// Create state and render
WaveTextState state = new WaveTextState();
wave.render(area, buffer, state);

// Advance animation each frame
state.advance();
```

Configuration options:

- `color(Color)` - Base color of the text
- `dimFactor(double)` - Brightness of dim portion (0.0-1.0, default 0.3)

- `peakWidth(int)` - Width of the wave peak in characters (default 3)
- `peakCount(int)` - Number of peaks in the wave (default 1)
- `speed(double)` - Animation speed multiplier (default 1.0)
- `mode(Mode)` - `LOOP` (continuous) or `OSCILLATE` (back-and-forth)
- `inverted(boolean)` - If true, bright peak on dim text; if false, dark shadow on bright text

CSS properties:

- `color` - Base text color
- `wave-dim-factor` - Brightness of dim portion
- `wave-peak-width` - Width of wave peak
- `wave-peak-count` - Number of peaks
- `wave-speed` - Animation speed multiplier

Input Widgets

TextInput

A single-line text input field:

```
// Create state
TextInputState state = new TextInputState();

// Or with initial value
TextInputState stateWithValue = new TextInputState("initial text");

// Build widget
TextInput textInput = TextInput.builder()
    .placeholder("Enter your name...")
    .cursorStyle(Style.EMPTY.reversed())
    .block(Block.builder()
        .title("Name")
        .borders(Borders.ALL)
        .build())
    .build();

textInput.render(area, buffer, state);
```

```
// Handle key events
if (event.code() == KeyCode.CHAR) {
    state.insert(event.character());
} else if (event.code() == KeyCode.BACKSPACE) {
    state.deleteBackward();
} else if (event.code() == KeyCode.DELETE) {
    state.deleteForward();
} else if (event.code() == KeyCode.LEFT) {
    state.moveCursorLeft();
} else if (event.code() == KeyCode.RIGHT) {
    state.moveCursorRight();
}
```

State methods:

- `state.text()` - Get current text
- `state.setText(text)` - Set text
- `state.insert(char)` - Insert character at cursor
- `state.deleteBackward()` - Backspace
- `state.deleteForward()` - Delete
- `state.moveCursorLeft()` / `moveCursorRight()` - Move cursor
- `state.clear()` - Clear all text

TextArea

A multi-line text input field with scrolling and optional line numbers:

```
// Create state
TextAreaState state = new TextAreaState();

// Or with initial content
TextAreaState stateWithContent = new TextAreaState("Line 1\nLine 2\nLine 3");

// Build widget
TextArea textArea = TextArea.builder()
    .placeholder("Enter text...")
    .showLineNumbers(true)
    .cursorStyle(Style.EMPTY.reversed())
```

```
        .lineNumberStyle(Style.EMPTY.dim())
        .block(Block.builder()
            .title("Editor")
            .borders(Borders.ALL)
            .build())
        .build();

    // Render with cursor visible
    textArea.renderWithCursor(area, buffer, state, frame);

    // Handle key events
    if (event.code() == KeyCode.CHAR) {
        state.insert(event.character());
    } else if (event.code() == KeyCode.ENTER) {
        state.insert('\n');
    } else if (event.code() == KeyCode.BACKSPACE) {
        state.deleteBackward();
    } else if (event.code() == KeyCode.UP) {
        state.moveCursorUp();
    } else if (event.code() == KeyCode.DOWN) {
        state.moveCursorDown();
    }
```

State methods:

- `state.text()` - Get full text content
- `state.setText(text)` - Set text content
- `state.lineCount()` - Get number of lines
- `state.getLine(row)` - Get text of specific line
- `state.insert(char)` / `state.insert(string)` - Insert at cursor
- `state.deleteBackward()` / `state.deleteForward()` - Delete characters
- `state.moveCursorUp()` / `moveCursorDown()` / `moveCursorLeft()` / `moveCursorRight()` - Move cursor
- `state.moveCursorToLineStart()` / `moveCursorToLineEnd()` - Jump to line boundaries
- `state.moveCursorToStart()` / `moveCursorToEnd()` - Jump to document boundaries
- `state.scrollUp(amount)` / `state.scrollDown(amount, visibleRows)` - Manual scrolling
- `state.clear()` - Clear all content

CSS properties:

- `cursor-color` - Cursor highlight color
- `placeholder-color` - Placeholder text color
- `line-number-color` - Line number gutter color

Checkbox

A toggleable checkbox widget displaying checked/unchecked state:

```
// Create state
CheckboxState state = new CheckboxState();       // unchecked
CheckboxState stateChecked = new CheckboxState(true);   // checked

// Build widget with default style [x] / [ ]
Checkbox checkbox = Checkbox.builder().build();

// Custom symbols
Checkbox checkboxCustom = Checkbox.builder()
    .checkedSymbol("[v]")
    .uncheckedSymbol("[ ]")
    .checkedColor(Color.GREEN)
    .uncheckedColor(Color.DARK_GRAY)
    .build();

checkbox.render(area, buffer, state);

// Handle input
if (event.code() == KeyCode.ENTER || (event.code() == KeyCode.CHAR && event.charac
    state.toggle();
}
```

State methods:

- `state.isChecked()` / `state.value()` - Get checked state
- `state.setChecked(boolean)` / `state.setValue(boolean)` - Set state
- `state.toggle()` - Toggle and return new value

CSS properties:

- `checkbox-checked-symbol` - Symbol when checked (default: "[x]")
- `checkbox-unchecked-symbol` - Symbol when unchecked (default: "[ ]")
- `checkbox-checked-color` - Foreground color when checked
- `checkbox-unchecked-color` - Foreground color when unchecked

Toggle

A toggle switch widget with two display modes:

```
// Create state
ToggleState state = new ToggleState();       // off
ToggleState stateOn = new ToggleState(true);   // on

// Single symbol mode (default): [ON ] or [OFF]
Toggle toggle = Toggle.builder()
    .onSymbol("o ON ")
    .offSymbol("o OFF")
    .onColor(Color.GREEN)
    .build();

// Inline choice mode: o Yes / o No
Toggle toggleInline = Toggle.builder()
    .inlineChoice(true)
    .onLabel("Yes")
    .offLabel("No")
    .selectedIndicator("o")
    .unselectedIndicator("o")
    .selectedColor(Color.GREEN)
    .unselectedColor(Color.DARK_GRAY)
    .build();

toggle.render(area, buffer, state);

// Handle input
if (event.code() == KeyCode.ENTER || (event.code() == KeyCode.CHAR && event.charac
    state.toggle();
}
```

State methods:

- `state.isOn()` / `state.value()` - Get on/off state
- `state.setOn(boolean)` / `state.setValue(boolean)` - Set state
- `state.toggle()` - Toggle and return new value

CSS properties (single mode):

- `toggle-on-symbol` - Symbol when on (default: "[ON]")
- `toggle-off-symbol` - Symbol when off (default: "[OFF]")
- `toggle-on-color` - Foreground color when on
- `toggle-off-color` - Foreground color when off

CSS properties (inline choice mode):

- `toggle-selected-indicator` - Selected option indicator (default: "●")
- `toggle-unselected-indicator` - Unselected option indicator (default: "○")
- `toggle-selected-color` - Color for selected option
- `toggle-unselected-color` - Color for unselected option

Select

An inline select widget showing the current selection with navigation indicators:

```
// Create state with options
SelectState state = new SelectState("Option A", "Option B", "Option C");

// Build widget: < Option A >
Select select = Select.builder()
    .leftIndicator("< ")
    .rightIndicator(" >")
    .selectedColor(Color.CYAN)
    .indicatorColor(Color.DARK_GRAY)
    .build();

select.render(area, buffer, state);

// Handle navigation
if (event.code() == KeyCode.LEFT) {
    state.selectPrevious();  // wraps around
} else if (event.code() == KeyCode.RIGHT) {
```

```
    state.selectNext();      // wraps around
}
```

State methods:

- `state.selectedValue()` / `state.value()` - Get selected option text
- `state.selectedIndex()` - Get selected index
- `state.options()` - Get all options
- `state.optionCount()` - Get number of options
- `state.selectIndex(int)` - Select by index
- `state.selectNext()` / `state.selectPrevious()` - Navigate (wraps)
- `state.selectFirst()` / `state.selectLast()` - Jump to ends
- `state.setOptions(…)` - Replace available options

CSS properties:

- `select-left-indicator` - Left navigation indicator (default: "< ")
- `select-right-indicator` - Right navigation indicator (default: " >")
- `select-selected-color` - Foreground color for selected value
- `select-indicator-color` - Foreground color for indicators

Utility Widgets

Scrollbar

Visual indicator of scroll position:

```
Scrollbar scrollbar = Scrollbar.builder()
    .orientation(ScrollbarOrientation.VERTICAL_RIGHT)
    .thumbStyle(Style.EMPTY.fg(Color.CYAN))
    .trackStyle(Style.EMPTY.fg(Color.DARK_GRAY))
    .build();

ScrollbarState state = new ScrollbarState();
```

```
state.contentLength(100);
state.viewportContentLength(20);
state.position(scrollPosition);

scrollbar.render(area, buffer, state);
```

Logo

Renders the Tamboui logo for use in help or about screens:

```
// Default tiny logo (2 lines)
Logo logo = Logo.tiny();
logo.render(area, buffer);

// Or the normal size (4 lines)
Logo logoNormal = Logo.of(Logo.Size.NORMAL);
logoNormal.render(area, buffer);
```

Available sizes:

- `Logo.Size.TINY` - 2-line compact logo using braille characters
- `Logo.Size.NORMAL` - 4-line logo using box-drawing characters

ErrorDisplay

Displays exception information with a formatted stack trace:

```
// Quick creation from exception
ErrorDisplay display = ErrorDisplay.from(exception);

// Or with customization
ErrorDisplay displayCustom = ErrorDisplay.builder()
    .error(exception)
    .title(" CRASH ")
    .footer(" Press 'q' to quit, arrows to scroll ")
    .borderColor(Color.RED)
    .scroll(scrollOffset)
    .build();
```

```
display.render(area, buffer);

// Handle scrolling
int totalLines = display.lineCount();
if (event.code() == KeyCode.DOWN) {
    scrollOffset = Math.min(scrollOffset + 1, totalLines - visibleHeight);
} else if (event.code() == KeyCode.UP) {
    scrollOffset = Math.max(0, scrollOffset - 1);
}
```

The widget displays:

- Exception type in red and bold
- Error message
- Full stack trace (scrollable)
- Footer with instructions

Clear

Clears an area by filling it with a style. Useful for widget layering:

```
// Clear is a singleton widget with no style support
Clear.INSTANCE.render(area, buffer);
```

Using Widgets with Toolkit DSL

The Toolkit DSL provides fluent factories for all widgets:

```
// Text
text("Hello").bold().cyan();

// Panel (Block)
panel("Title", text("child")).rounded().borderColor(Color.CYAN);

// List (manages its own state)
list("Item 1", "Item 2", "Item 3").highlightColor(Color.YELLOW).autoScroll();
```

```
// Table
table()
    .header("Name", "Age")
    .row("Alice", "30")
    .state(tableState);

// Gauge
gauge(0.75).label("Loading...").gaugeColor(Color.GREEN);

// Sparkline
sparkline(1, 4, 2, 8, 5).color(Color.CYAN);

// Spinner (manages its own state, advances each render)
spinner().cyan();
spinner(SpinnerStyle.LINE, "Loading...").green();

// Tree (expand/collapse, keyboard navigation, lazy loading)
tree(
    TreeNode.of("Root",
        TreeNode.of("Child 1"),
        TreeNode.of("Child 2"))
).highlightColor(Color.CYAN).rounded().scrollbar();

// TextInput
textInput(inputState).placeholder("Enter name...");

// Layout
row(left, right).spacing(1);
column(top, middle, bottom).spacing(1);
```

For layout DSL examples (columns, grid, dock, stack, flow), see Layouts Reference.

See API Levels for more details on the Toolkit DSL.

PREVIOUS

Core Concepts

NEXT

Demo Gallery